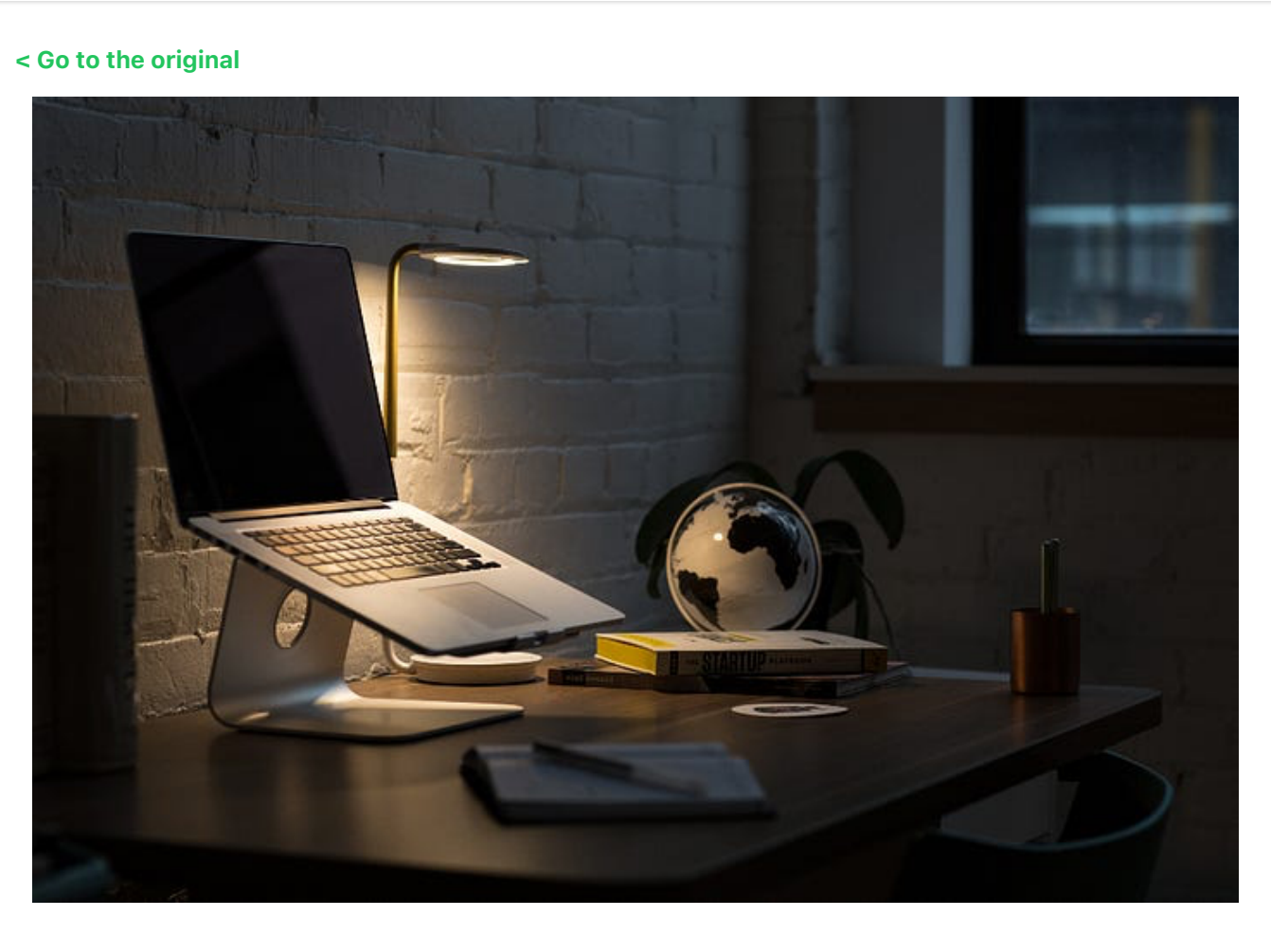
< Go to the original



MacOS: Building the Perfect Development Machine in 2025

How I Transformed My MacBook Into a Development Powerhouse

Promise Chukwuenyem
Follow
Level Up Coding a11y-light ~11 min read · January 14, 2025 (Updated: January 28, 2025) · Free: No

The Start of My Journey

"Where's my terminal? How do I even install packages?"

These were my first words in 2021 as I turned on my MacBook Pro 2018. Coming from years of doing development on Linux on my older computer, I felt a bit lost. While the UI for macOS looked sleek, I wanted the familiar feel that my development environment has always had.

If you're reading this, you might be in the same situation whether you're migrating from Linux like I did, coming from Windows, or just starting your development journey with a new MacBook. Let me share with you how I turned my initial confusion into a love story with macOS.

Why This Guide Exists

When I set up my development environment on my MacBook Pro I had to:

- Watch countless YouTube videos
- Read through dozens of blogs
- Find solutions for various setup issues
- Test various tools and configurations

I have outlined all this here so that you yourself don't have to go through the same hassle. Let's build a powerful development environment together.

Part 1: Building Our Foundation

Remember building a house? You need a solid foundation first. On MacOS, we start with the essential building blocks.

Xcode Command Line Tools

First, we have to install command-line tools for Xcode. Although macOS has Unix roots, not all development tools are included. Here's how you install them:

Copy

```
xcode-select --install
```

> **Pro tip**: Do this step first. I spent several hours fixing installation issues due to skipping this step at first.

Homebrew — Your Package Manager

If you have a Linux background you will really miss `apt-get` on Mac OS X. Until I found Homebrew which is essentially the App Store on command line. Think of Homebrew as `apt-get`'s cool cousin. To install here are the steps:

Copy

```
/bin/bash -c "$(curl -fsSL <https://raw.githubusercontent.com/Homebrew/inst
```

And remember post install use:

Copy

```
brew doctor
```

This checks that all is properly set up. Believe me, this little check saves so many headaches troubleshooting later on.

Part 2: Modern Terminal Setup

We will be spending a lot of our time here, so let's make it powerful and pretty.

Terminal Emulators

I love having options in terminal emulators. I use the default MacOS Terminal app pretty regularly, it's actually pretty great, but I've also got these installed for when specific situations arise:

Copy

```
# Install both, you'll thank me later for the options
brew install --cask iterm2    # Full-featured terminal
brew install --cask warp      # Modern, AI-powered terminal
```

iTerm2 shines when I need to monitor several services in split panes. In general, though, Warp is quite nice to have for its AI features in suggesting commands. But honestly? The default Terminal app handles most of my daily DevOps tasks just fine. It's fast, reliable, and always there when you need it.

> **Pro tip**: Set up your default Terminal app first, you might find it's all you'll ever need. The others are nice to have in your toolbox for those special situations.

Must-Have Command Line Tools

These modern tools will greatly enhance your command line experience:

Copy

```
# Modern CLI tools
brew install \
  bat          # Better cat with syntax highlighting
  exa          # ls modern replacement
  fd           # Easy to use alternative to find
  ripgrep      # Extremely fast grep
  fzf          # Fuzzy finder
  jq           # JSON processor
  yq           # YAML processor
  delta        # Better diff
  zoxide       # Smarter cd command
  mcfly        # Better history search
  starship     # Cross-shell prompt
```

ZSH Configuration

MacOS uses **zsh** by default since Catalina. Let's make it powerful:

Install latest zsh:

Copy

```
brew install zsh
chsh -s /usr/local/bin/zsh
```

Install Oh My Zsh:

Copy

```
sh -c "$(curl -fsSL <https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/mast
```

Install PowerLevel10k theme:

Copy

```
git clone <https://github.com/romkatv/powerlevel10k.git> ${ZSH_CUSTOM:-$HOM
```

Install necessary plugins:

Copy

```
# Auto-suggestions
git clone <https://github.com/zsh-users/zsh-autosuggestions> ${ZSH_CUSTOM:-

# Syntax highlighting
git clone <https://github.com/zsh-users/zsh-syntax-highlighting.git> ${ZSH_
```

Configure `~/.zshrc`:

Copy

```
# Theme
ZSH_THEME="powerlevel10k/powerlevel10k"

# Plugins
plugins=(
    git
    docker
    kubectl
    aws
    terraform
    python
    node
    ruby
    vscode
    zsh-syntax-highlighting
    zsh-autosuggestions
)
# Modern CLI aliases
alias ls='exa --icons --git'
alias ll='exa -l --icons --git'
alias cat='bat'
alias find='fd'
alias grep='rg'
alias cd='z'
# Initialize starship
eval "$(starship init zsh)"
```

Part 3: Building Your Dream Development Setup

Editor Setup

VS Code has been a constant companion during my Linux days. I still remember setting it up on my first day with the MacBook, that familiar interface made me feel at home immediately. Here is how I set it up:

Copy

```
brew install --cask visual-studio-code
```

These are the extensions that I use on a daily basis that have truly improved my workflow:

Copy

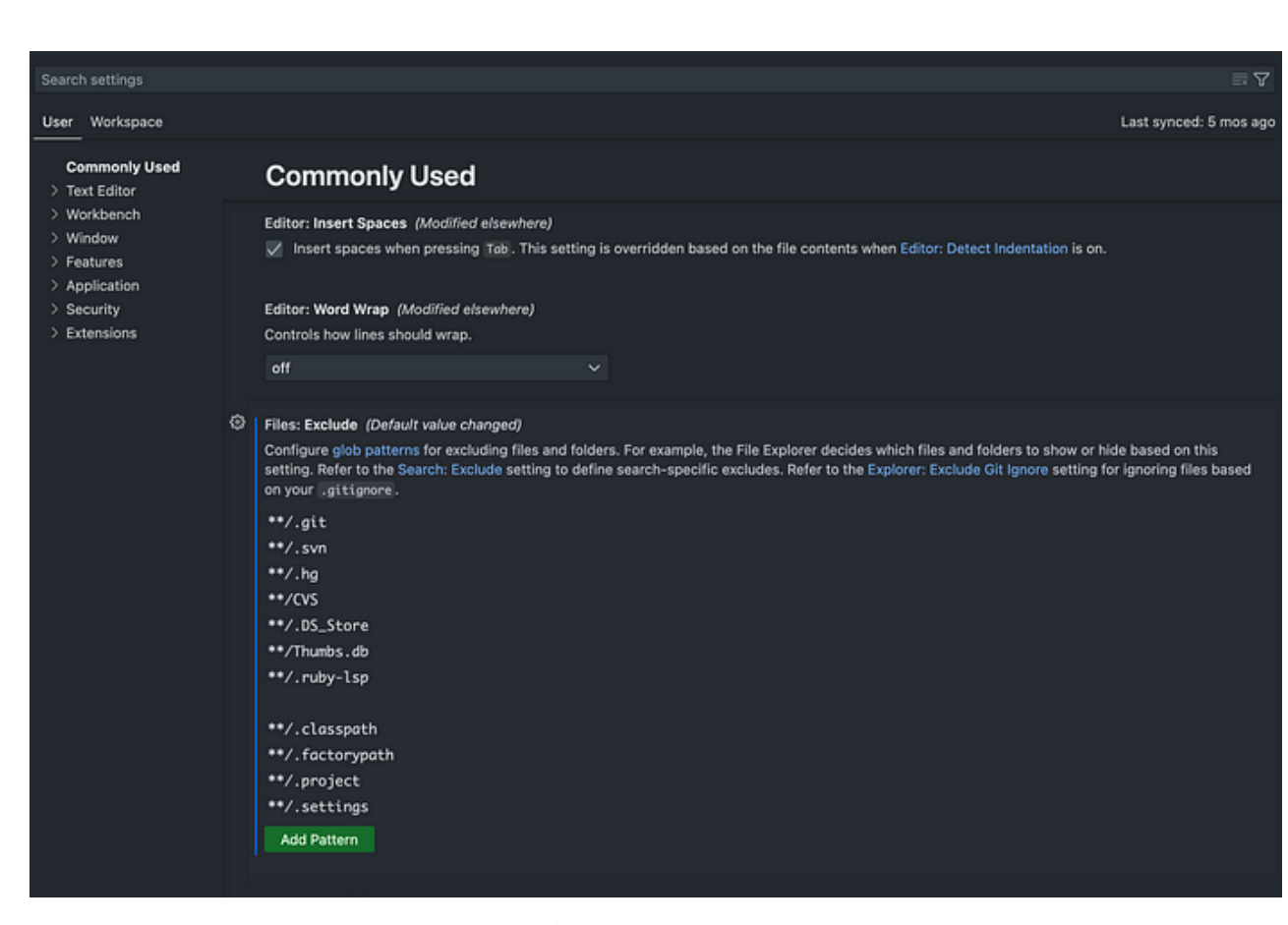
```
# AI Assistance & Productivity
code --install-extension github.copilot        # AI pair programmer
code --install-extension github.copilot-chat   # AI coding assistant
code --install-extension tabnine.tabnine-vscode # AI code completion

# Cloud & Infrastructure
code --install-extension hashicorp.terraform        # Terraform support
code --install-extension ms-azuretools.vscode-docker # Docker integration
code --install-extension ms-kubernetes-tools.vscode-kubernetes-tools # K8s

# Development Essentials
code --install-extension eamodio.gitlens          # Git superpowers
code --install-extension redhat.vscode-yaml       # YAML support
code --install-extension esbenp.prettier-vscode   # Code formatting
code --install-extension ms-python.python         # Python support
code --install-extension golang.go                # Go support
code --install-extension shopify.ruby-lsp         # Ruby language server
code --install-extension castwide.solargraph      # Ruby intellisense
code --install-extension misogi.ruby-rubocop      # Ruby code analyzer
code --install-extension hridoy.rails-snippets    # Rails snippets

# API & Testing
code --install-extension rangav.vscode-thunder-client # API testing
```

> **Pro tip**: Rather than installing every extension you think you might want, let these few get you started, install more only when you find yourself needing the exact functionality repeatedly. Your VS Code will thank you with faster load times.

Speaking of themes, I was a die-hard Dracula theme fan until I discovered GitHub Dark. Try coding at 2 AM (we've all been there), and you'll understand why I switched. Your eyes will thank you!

Once more thing, VS Code (and macOS in general) hides files and folders that start with a dot (.), including the `.git` folder. Here is how to view the `.git` folder:

- Press `Cmd + Shift + P` to open the command palette
- Type **preferences: Open Settings (UI)**
- Remove `**/.git` from the list of **Files: Exclude**



Editor: Insert Spaces (Modified elsewhere)
☑ Insert spaces when pressing Tab. This setting is overridden based on the file contents when Editor: Detect Indentation is on.

Editor: Word Wrap (Modified elsewhere)
Controls how lines should wrap.
off

Files: Exclude (Default value changed)
Configure glob patterns for excluding files and folders. For example, the File Explorer decides which files and folders to show or hide based on this setting. Refer to the Search: Exclude setting to define search-specific excludes. Refer to the Explorer: Exclude Git Ignore setting for ignoring files based on your .gitignore.

```
**/.git
**/.svn
**/.hg
**/CVS
**/.DS_Store
**/Thumbs.db
**/.ruby-lsp

**/.classpath
**/.factorypath
**/.project
**/.settings
```

Screenshot of the VS Code Open Settings (UI)

Container Setup

Docker was one of my daily drivers on Linux, and I brought that love to Mac. I use Docker Desktop as my main container tool:

Copy

```
# My primary container setup
brew install --cask docker
```

But here's something cool I discovered along the way, if you're looking for a lighter alternative, especially when your MacBook's fan starts sounding like a jet engine, try Colima:

Copy

```
brew install colima
brew install docker
brew install docker-compose
```

Version Management

Let's talk about managing different versions of Node.js, Python, and Ruby. Rather than telling you the "perfect" setup, let me share what actually works for me:

Node.js Setup

Copy

```
# My Node.js home
brew install nvm
```

And here is my `~/.zshrc` setup that I use daily:

Copy

```
# The magic sauce
export NVM_DIR="$HOME/.nvm"
[ -s "/usr/local/opt/nvm/nvm.sh" ] && . "/usr/local/opt/nvm/nvm.sh"
```

> **Pro tip**: After multiple "node command not found" errors, I have learned to run this right away after installing any new Node version:

Copy

```
nvm use 16.4.2           # Pick your version
nvm alias default 16.4.2  # Make it stick
```

Python Setup

Copy

```
# Install pyenv
brew install pyenv

# List available Python versions
pyenv install --list

# Install latest Python version
pyenv install 3.12.0

# Set global Python version
pyenv global 3.12.0

# Verify installation on a new terminal
python --version  # Should show Python 3.12.0

# Add to ~/.zshrc
eval "$(pyenv init --path)"
eval "$(pyenv init -)"
```

Ruby Setup

Copy

```
# Install rbenv and ruby-build
brew install rbenv ruby-build

# Add to ~/.zshrc
eval "$(rbenv init -)"

# List available Ruby versions
rbenv install -l

# Install latest Ruby version
rbenv install 3.3.0
rbenv global 3.3.0
OR
rbenv local 3.3.0

# Verify installation on a new terminal
ruby -v  # Should print Ruby 3.3.0

# Skip documentation installation for gems
echo "gem: --no-document" >> ~/.gemrc
```

> **Pro tip**: For Ruby projects, I always create a `.ruby-version` file in the project root. This automatically switches Ruby versions when you change directories:

Copy

```
echo "3.2.0" > .ruby-version
```

This setup has served me well for both Rails development and Ruby-based DevOps tools like Chef and Puppet.

Communication Setup

Nothing is different for my Mac setup in terms of communication tools. Slack, Discord, Zoom were part of my everyday routine in Linux and followed me over to Mac:

Copy

```
# My communication must-haves
brew install --cask \
  slack \
  discord \
  zoom \
  microsoft-teams
```

Database Setup

Every developer needs his/her databases, and here is my setup after lots of trial and error:

Copy

```
# Database must-haves
brew install postgresql@15
brew install redis
brew install mongodb-community
```

For Database management, I went with DBeaver Community:

Copy

```
brew install --cask dbeaver-community
```

> **Pro tip**: If you are dealing with several types of databases, DBeaver saves tons of headaches. I used to have one tool for PostgreSQL, MongoDB and MySQL. Now? One to rule them all.

Remote Work Tools

RustDesk has turned into one of the most critical tools in my DevOps toolbox. This open-source alternative to TeamViewer enables jumping into the MacBook from anywhere, managing running containers, or managing Kubernetes clusters and fixing CI/CD pipelines. It is perfect for that time when I really want to use my development environment from my iPad during late-night incident responses or just away from my desk.

Copy

```
# Remote work
brew install --cask rustdesk     # Free remote desktop that actually works
```

Clipboard Manager

Remember that time you copied something important, then copied something else, and lost the first thing? Never again. Clipy keeps a history of your clipboard. It's like having an undo button for your copies. My favourite keyboard shortcut? `Cmd + Shift + V` to see my clipboard history.

Copy

```
brew install --cask clipy
```

System Monitoring

Stats has become my go-to system monitor:

Copy

```
brew install --cask stats
```

Here is how I have it configured:

1. Show CPU, Memory, Disk, and Network in the menu bar
2. Set update frequency to 2 seconds
3. Enable battery time remaining
4. Show network speeds

A quick glance at my menu bar now gives me everything I need to know about my system's health.

Window Management

Coming from Linux's tiling window managers, I was very apprehensive about Mac's way of handling its windows. Moom changed this, it's just like having the powers of Linux window management with all the polish of Mac:

Copy

```
brew install --cask moom
```

Some of the most used Moom shortcuts from me are:

- `Ctrl + [Arrow]` for half-screen layouts
- `Ctrl + Enter` for full screen
- `Ctrl + Shift + [Arrow]` for quarter-screen layouts

Updating Everything

MacUpdater has saved me hours and hours:

Copy

```
brew install --cask macupdater
```

Rather than open each app and check if an update is available, MacUpdater does it for you. It even picks up updates for applications installed outside the Mac App Store and Homebrew.

> **Pro tip**: Configure it to check for updates daily but to not auto-install. Trust me, you'll want to decide when your development tools update.

Note-Taking and Documentation

After trying what seemed like each and every note-taking app under the sun, Notion was the one that finally stuck. It's become my second brain:

Copy

```
brew install --cask notion
```

I use it for:

- Project documentation
- Code snippets
- Meeting notes
- Personal wiki

The icing on the cake? The code blocks support syntax highlighting for pretty much every language I use.

Simplifying Git Workflow

Even though I use inbuilt git features in VS Code, these below tools have made my git life a lot easier:

Copy

```
# Git helpers I actually use
```

```
brew install gh         # GitHub CLI is a game-changer
brew install lazygit    # For those complex git operations
```

Special shoutout to the GitHub CLI. It's nice to create PRs directly from the terminal, among other minor GitHub operations.

Security Tools

Security is important, but it shouldn't make your life harder. Here's my practical security setup:

Copy

```
# Security tools I actually use
brew install \
  age \       # Modern encryption that's not a pain to use
  sops \      # Secrets management that makes sense
```

I once accidentally committed AWS keys to GitHub. Since then, I use these pre-commit hooks to prevent disasters:

Copy

```
# Install pre-commit
brew install pre-commit
# Create .pre-commit-config.yaml
cat << EOF > .pre-commit-config.yaml
repos:
-   repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v5.0.0
    hooks:
    -   id: detect-aws-credentials
    -   id: detect-private-key
EOF
```

Part 4: Making Your Mac Work For You

While I love Homebrew, some App Store apps have become integral to my workflow:

My Daily Drivers

- **Be Focused**: My Pomodoro timer of choice. Tried terminal-based timers, but having it in the menu bar just works better.
- **Microsoft To Do**: Keeps my tasks synced across devices. Simple, fast, and it just works.
- **Bitwarden**: Password management that gets out of the way. Autofill is super smooth, and it gets along swimmingly with terminal applications.
- **AppCrypt**: Ever opened up Twitter "just to check for a minute" while you were coding? Yeah. This app locks it up during worktime.

File Management Heroes

- **Folder Tidy**: Life is messy but this app doesn't judge, haha, it just cleans. Run it once a week on your Downloads folder.
- **AVG Cleaner**: Thought I'd share since it was helpful for me to resolve issues with duplicate files. Safer than the more dangerous `rmlint`.

Automation using Keyboard Maestro

If ever you need a tool to automate those small repetitive tasks that take up your time, Keyboard Maestro is here to save you. It is like scripting your whole Mac environment, window management to application workflows, anything, triggered by the press of a customizable hotkey.

Copy

```
# Install Keyboard Maestro
brew install — cask keyboard-maestro
```

The most used automation which saves me hours every week includes:

1. One hotkey to set up my whole development environment: - Opens VS Code with specific projects - Launches Terminal with splits for different services - Spins up the needed Docker containers - Opens respective documentation in browser tabs
2. Automatic windows arrangement when connecting/disconnecting monitors: - When I dock/undock, automatically organize windows - The IDEs, terminals, and browsers keep their perfect spot - Adapts layouts whether I'm on one or two screens
3. Text expansions for common code snippets: - Quick insertion of Dockerfile templates - Git commit message formats - Common cloud resource configurations - Kubernetes manifest structures

Let me tell you, after you start making your workflow automated with Keyboard Maestro, you will probably wonder how you ever worked without it.

Maintenance

Here is a simple weekly maintenance script that I run to make sure everything is running okay.

Copy

```
echo "🧹 Starting weekly maintenance..."

# Update package managers
echo "📦 Updating Homebrew..."
brew update && brew upgrade
brew upgrade --cask --greedy
brew cleanup

# Update App Store apps
echo "🏪 Updating App Store apps..."
mas upgrade

# Clean up Docker
echo "🐳 Cleaning Docker..."
docker system prune -f --volumes

# Clean up downloads folder
echo "🗑️ Cleaning Downloads..."
find ~/Downloads -mtime +30 -exec mv {} ~/.Trash \;

echo "✨ Maintenance complete!"
```

> **Pro tip:** You can automate this using Keyboard Maestro to run say every Saturday or Sunday night before a new work week starts.

Terminal Workflows

Some simple terminal configurations that I use everyday:

Copy

```
# Add to ~/.zshrc
alias dev="cd ~/Development && code ."
alias dotfiles="cd ~/.dotfiles && code ."
alias cleanup="brew cleanup && docker system prune -f"
```

What's Next

The Mac development environment is one that continuously changes. Keep an eye out for the following:

- More AI-driven development tools
- Even more cloud-native development environments
- Greater and deeper integration of terminal and GUI tools

Conclusion

Writing this article with my MacBook Pro M3, I can confidently say that switching from Linux to Mac was more than just using a new system, it changed how I work as a Developer/DevOps Engineer. After three years, I'm happy I made the move. My Mac gives me the best of both worlds: the power of Unix commands I loved in Linux, plus great tools that make my daily work easier.

> **Remember:** There's no one-size-fits-all setup. Use what works best for you from this guide and build your own perfect environment.

You can follow me here on Medium for updates on my future articles. I will continuously update this article with newer findings.

Connect With Me

You can reach out to me and get to know more about me through other social networks: - Twitter: PromisePreston - LinkedIn: Promise Chukwuenyem - Personal Website: Personal Portfolio

Feel free to comment below if you have any questions or have an even better way of accomplishing something.